my slower integration tests from my faster unit tests. For those things, it works great.

You'll sometimes hear that Cucumber allows for nondeveloper members of your team to participate in the acceptance-testing process because Cucumber is natural language–like. My experiences in that regard are mixed. I've had some success with writing Cucumber scenarios on my own and giving them to managers or clients for approval. Going the other direction, the limiting factor in my experience is not the syntax, but rather experience with how to specify requirements. That's tricky for everybody.

Some tips for better Cukeing:

- Write the scenario in natural language that defines the system's behavior from the user perspective; smooth out details in the step definition.
- Avoid anything in the feature file that looks like code or data—that includes CSS selectors and database attributes.
- Keep step definitions simple.
- Don't worry about duplicating bits of step logic. Prefer multiple simple steps over one big step with complex logic.
- Specify what isn't on the page; that's often as important as specifying what is.
- Worry about implementation details in the unit tests. The suggestions about what is an integration test and what is a unit test also apply here.
- Validate against user-visible pages rather than database internals. (Sometimes this means going to an admin page or similar to validate that a change has taken place.)

What You've Done

In this chapter you used Capybara to drive Rails system tests that cover an entire user interaction, then used Cucumber to show an alternative way to describe requirements for testing.

I'll talk more about these tools in future chapters. In Chapter 9, *Testing JavaScript: Integration Testing*, on page 181, I'll cover how Capybara and Cucumber can be attached to drivers that run the tests against a browser engine that executes JavaScript, allowing for client-side actions to be integration-tested. In Chapter 14, *Testing External Services*, on page 285, I'll talk about integration tests that might need to touch a third-party service. And in Chapter 16, *Running Tests Faster and Running Faster Tests*, on page 319, I'll talk about how to optimize the command-line execution of both tools so as to speed up your feedback loop.